

VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

School of Computer Science and Engineering (SCOPE)

B.Tech Computer Science and Engineering

PROJECT REVIEW 3 (FINAL REVIEW)

PROJECT REPORT

IN-PROCESS ASGI CACHING MIDDLEWARE & SINGLE-FLIGHT CACHE STAMPEDE MITIGATOR

High-Concurrency Request Coalescing, Stateless HMAC-SHA256 JWT Gatekeeping, and Multi-Tenant Redis Caching over PostgreSQL

Submitted by

C S Akshay Khanna Reddy

Registration Number: 25BCE1113

Adarsh Varshan V N

Registration Number: 25BCE1055

Raghav Manickam

Registration Number: 25BCE1978

November 2026

INDEX / TABLE OF CONTENTS

Sec #	Main Section Heading	Subheading (One Subheading Per Page)
1	Title, Team Details, and Abstract	1.1 Project Metadata & Abstract
2	Keywords	2.1 Technical Index Terms
3	Introduction, Problem Statement, Objectives, and Scope	3.1 Introduction
3	Introduction, Problem Statement, Objectives, and Scope	3.2 Problem Statement
3	Introduction, Problem Statement, Objectives, and Scope	3.3 Objectives
3	Introduction, Problem Statement, Objectives, and Scope	3.4 Project Scope
4	Existing Systems	4.1 Overview of Industry Caching Architectures
4	Existing Systems	4.2 Architectural Comparison Matrix
5	Requirements and Proposed System Architecture	5.1 System Requirements Specification
5	Requirements and Proposed System Architecture	5.2 Proposed 5-Layer System Architecture
5	Requirements and Proposed System Architecture	5.3 Request Execution Pipeline Matrix
6	Schema	6.1 Single-Entity High-Cardinality Schema Justification
6	Schema	6.2 PostgreSQL DDL & 50,000-Row Seeding Code (postgres/init.sql)
6	Schema	6.3 Schema Data Dictionary
6	Schema	6.4 Relational-to-Redis Key Namespacing Schema
7	Implementation Details: DBMS, Application Technologies, and Database Connectivity	7.1 Layer 5 Container Orchestration Code (docker-compose.yml)
7	Implementation Details: DBMS, Application Technologies, and Database Connectivity	7.2 DBMS Connectivity, Healthchecks vs. Lifespan & os.getenv
7	Implementation Details: DBMS, Application Technologies, and Database Connectivity	7.3 FastAPI ASGI Middleware Source Code (middleware/main.py)
7	Implementation Details: DBMS, Application Technologies, and Database Connectivity	7.4 Streamlit Telemetry & Simulator Code (frontend/dashboard.py)
7	Implementation Details: DBMS, Application Technologies, and Database Connectivity	7.5 Locust Workload Benchmark Code (locust/locustfile.py)
8	Implemented Features	8.1 Functional Capabilities Overview

9	Testing, Results, Screenshots, and Discussion	9.1 Locust Load Testing Configuration & Aggregate Results
9	Testing, Results, Screenshots, and Discussion	9.2 Longitudinal Streamlit Telemetry Progression
9	Testing, Results, Screenshots, and Discussion	9.3 Stampede Burst Simulator & Security Inspector Results
9	Testing, Results, Screenshots, and Discussion	9.4 Discussion of Verification Screenshots (1.png – 19.png)
10	Conclusion, Limitations, and Future Enhancements	10.1 Conclusion
10	Conclusion, Limitations, and Future Enhancements	10.2 Architectural Weaknesses and Known Limitations
10	Conclusion, Limitations, and Future Enhancements	10.3 Future Enhancements
11	References	11.1 IEEE-Style References

1. TITLE, TEAM DETAILS, AND ABSTRACT

1.1 Project Metadata, Team Responsibilities, and Abstract

Project Title: In-Process ASGI Caching Middleware & Single-Flight Cache Stampede Mitigator for High-Concurrency Microservices.

Team Member Name	Register Number	Primary Engineering Responsibilities
C S Akshay Khanna Reddy	25BCE1113	System Architecture Design, Multi-Scenario Locust Workload Modeling & Performance Benchmarking
Adarsh Varshan V N	25BCE1055	In-Process ASGI Middleware Pipeline, Single-Flight asyncio.Lock Coalescing, JWT Security & Streamlit Telemetry
Raghav Manickam	25BCE1978	PostgreSQL 50k Schema Engineering, Cache-Aside PUT Invalidation, Redis LRU Tuning & Docker Orchestration

Abstract:

Modern high-throughput microservices face severe performance degradation and cascading database failures when subjected to skewed read traffic and thundering herd cache stampedes. While external reverse proxies such as NGINX or Varnish offload static responses, they lack native visibility into application-layer state, requiring auxiliary network hops or scripting for fine-grained authentication and dynamic multi-tenant key namespacing. This project implements an in-process Asynchronous Server Gateway Interface (ASGI) caching middleware built natively inside a FastAPI and Uvicorn runtime. Operating directly within the Python event loop, the middleware intercepts inbound HTTP scopes, validates HMAC-SHA256 JSON Web Tokens in CPU memory, enforces tenant-isolated cache key namespacing, and coordinates concurrent cache misses using an in-process single-flight asyncio.Lock mechanism. Validated across a 50,000-record PostgreSQL catalog using Locust and Streamlit, the system coalesces 50-request cold stampedes into a single database query while delivering sub-2ms cached responses and zero failures.

2. KEYWORDS

2.1 Technical Index Terms

Keywords: Asynchronous Server Gateway Interface (ASGI), Cache Stampede Mitigation, Thundering Herd Problem, Single-Flight Request Coalescing, Double-Checked Locking (asyncio.Lock), Distributed In-Memory Caching, Redis Allkeys-LRU Eviction, PostgreSQL Connection Pool Exhaustion (asyncpg), Stateless HMAC-SHA256 JWT Authentication, Pre-Cache Role-Based Access Control (RBAC), Dynamic Multi-Tenant Key Namespacing, Cache-Aside Invalidation, Eventual Consistency, Microservice Container Orchestration.

3. INTRODUCTION, PROBLEM STATEMENT, OBJECTIVES, AND SCOPE

3.1 Introduction

In contemporary cloud-native web architectures, read operations overwhelmingly outnumber write mutations—often following a highly skewed Zipfian distribution where a small subset of 'hot' catalog records accounts for the vast majority of client traffic. Relying exclusively on a relational database management system (RDBMS) such as PostgreSQL to serve every incoming HTTP read request introduces substantial disk I/O overhead, B-tree index traversal latency, and TCP connection pool contention. To achieve low-latency scalability, distributed in-memory key-value stores such as Redis are deployed alongside relational databases using the Cache-Aside architectural pattern.

However, traditional caching architectures suffer from two critical structural vulnerabilities under production load: (1) the Cache Stampede (or 'Thundering Herd') phenomenon when a popular cached record expires or starts cold under heavy concurrency, and (2) architectural decoupling between external reverse proxies and internal application security semantics. This project bridges that gap by embedding an application-aware, stampede-resilient caching proxy directly inside the Python ASGI execution loop.

3.2 Problem Statement

- **1. The Cache Stampede (Thundering Herd) Failure Mode:** Under standard Cache-Aside implementations, when a heavily requested key is absent from Redis (Cold Miss or TTL Expiration) and N concurrent requests arrive simultaneously, all N coroutines observe a cache miss at the exact same millisecond. Consequently, all N requests bypass the cache and execute identical, redundant SQL queries against PostgreSQL. This sudden spike saturates the database connection pool (asyncpg), spikes CPU utilization, and triggers cascading request timeouts.
- **2. Limitations of External Reverse Proxies (NGINX / Varnish):** Industry reverse proxies operate as separate networking tiers upstream of the application server. Because they only inspect raw HTTP byte streams, they cannot natively validate application-level cryptographic tokens (HMAC-SHA256 JWTs) or extract internal user claims without invoking external Lua plugins (OpenResty) or making additional network sub-requests (auth_request). Furthermore, every cache miss incurs an extra network hop (Client -> Proxy -> Application Server -> Redis/DB).
- **3. Multi-Tenant Cache Poisoning and Data Leakage Risks:** Standard URL-based caching keys (such as GET:/admin/analytics) fail to distinguish between authenticated tenants or privilege tiers. Without dynamic claim-aware key construction and pre-cache authorization checks, unauthorized users can probe cached endpoints or view data belonging to other administrative sessions.

3.3 Objectives

- **Objective 1 — In-Process ASGI Interception:** Design a custom FastAPI/Starlette ASGI middleware layer that intercepts HTTP requests prior to route handler execution and constructs raw JSON responses directly from Redis memory on cache hits.
- **Objective 2 — Single-Flight Stampede Coalescing:** Implement a per-key asynchronous mutex registry (asyncio.Lock) with double-checked Redis locking so that among N concurrent requests for a cold key, exactly 1 leader queries PostgreSQL while N-1 followers wait and read the newly populated cache.
- **Objective 3 — Zero-Hop Cryptographic Gatekeeping:** Execute stateless HMAC-SHA256 JWT verification and Role-Based Access Control (RBAC) inside CPU memory before generating Redis keys or opening database sockets.
- **Objective 4 — Multi-Tenant Key Isolation:** Synthesize composite cache keys incorporating HTTP method, URI path, user role, and subject ID (GET:/admin/analytics:role:admin:user:42) to guarantee zero cross-tenant data leakage.
- **Objective 5 — Cache-Aside Mutation & Eviction:** Implement atomic stock update operations (PUT /products/{id}) in PostgreSQL paired with immediate Redis key invalidation (DEL) and analyze the resulting consistency vs. latency trade-offs.
- **Objective 6 — Empirical Telemetry & Stress Validation:** Orchestrate a 5-container Docker environment featuring a 50,000-row PostgreSQL database, a 3-tab Streamlit live telemetry dashboard, and a weighted Locust load-testing harness.

3.4 Project Scope

The scope of this project encompasses the end-to-end design, container orchestration, and empirical benchmarking of a 5-service microservice stack (proxy_postgres, proxy_redis, proxy_middleware, proxy_frontend, and proxy_locust). The persistence layer models a high-cardinality enterprise hardware catalog of 50,000 records. The middleware is scoped to a single-process Uvicorn asynchronous event loop (--workers 1) to guarantee deterministic in-memory lock sharing across concurrent coroutines, trading multi-node distributed locking complexity and strict write serialization for ultra-low read latency (~1.28 ms to 16 ms) and complete single-instance stampede protection.

4. EXISTING SYSTEMS

4.1 Overview of Industry Caching Architectures

- **1. NGINX Reverse Proxy (ngx_http_proxy_module):** Written in C, NGINX sits upstream of backend application servers and caches raw HTTP responses on local disk or shared memory zones. While NGINX offers a proxy_cache_lock directive to limit concurrent upstream misses, its cache keys are derived from static HTTP primitives ($scheme$request_method$host$request_uri). Enforcing JWT verification or role-scoped key partitioning requires either commercial NGINX Plus modules, custom OpenResty Lua scripts, or an external auth_request sub-hop back to the Python application—negating the latency benefit of edge interception.
- **2. Varnish HTTP Accelerator:** Varnish uses the domain-specific Varnish Configuration Language (VCL) to manage in-memory caching and request coalescing (waitlists). Despite high raw throughput for public static content, Varnish introduces an additional infrastructure hop on every cache miss and cannot natively share in-process data structures, database connection pools, or Python cryptographic libraries with the backend service.
- **3. Envoy Cloud-Native Service Proxy:** Envoy provides L7 traffic routing and external authorization filters (ext_authz). However, deploying Envoy as a sidecar proxy requires maintaining separate YAML filter chains, external gRPC authorization servers, and out-of-process cache filters, adding substantial operational and memory overhead for application-specific data caching.
- **4. Naive Application-Level Cache-Aside (Unprotected FastAPI + Redis):** Many microservices query Redis directly inside individual route handlers without concurrency locks. While simple to implement, this approach offers zero protection against cache stampedes: when a key expires, dozens of concurrent coroutines simultaneously miss in Redis and flood PostgreSQL with duplicate queries.

4.2 Architectural Comparison Matrix

Architectural Dimension	External Proxy (NGINX / Varnish)	Naive App Cache-Aside (No Lock)	Proposed In-Process ASGI Middleware
Execution Boundary	Separate upstream OS process / container	Inside individual route handlers	In-process ASGI pipeline (Uvicorn event loop)
Network Hops on Cache Miss	3 Hops (Client -> Proxy -> App -> DB)	2 Hops (Client -> App -> Redis/DB)	2 Hops (Client -> App -> Redis/DB; zero proxy hop)
JWT Auth & RBAC Verification	Requires Lua scripting or auth_request hop	Executed per route after middleware	Zero-hop in-memory HMAC-SHA256 pre-cache check
Cache Key Granularity	Static URL / Header strings	Manual per-endpoint string formatting	Automated dynamic RBAC & tenant-scoped keys
Cache Stampede Protection	Coarse URI lock (proxy_cache_lock)	None (all concurrent misses hit DB)	Per-key asyncio.Lock + Double-Checked Redis read
Telemetry & Introspection	External log parsing required	Scattered route-level logging	Integrated ring-buffer & real-time KPI counters

5. REQUIREMENTS AND PROPOSED SYSTEM ARCHITECTURE

5.1 System Requirements Specification

Category	Component / Parameter	Specification & Configuration
Host Environment	Operating System & Virtualization	Ubuntu Linux (VirtualBox VM / Native Linux) or Windows Host with WSL2
Container Runtime	Docker & Compose Engine	Docker Engine v24.0+ and Docker Compose v2.20+ (Bridge Networking)
Origin Database	PostgreSQL Server	postgres:15-alpine (Port 5432, Persistent Volume pg_bench_data, 50,000 rows)
In-Memory Cache	Redis Server	redis:7-alpine (Port 6379, --maxmemory 256mb, --maxmemory-policy allkeys-lru)
Application Server	ASGI Framework & Runtime	Python 3.11-slim, FastAPI, Uvicorn (--host 0.0.0.0 --port 8000 --workers 1)
Database Driver	Asynchronous PostgreSQL Pool	asyncpg (Binary protocol pool: min_size=10, max_size=30 connections)
Observability UI	Telemetry Dashboard	Streamlit (Port 8501, 2s live polling, httpx AsyncClient burst simulator)
Load Testing Engine	Concurrency Benchmark Suite	Locust 2.x (Port 8089, weighted HttpUser coroutine swarm)

5.2 Proposed 5-Layer System Architecture

The proposed architecture organizes five containerized services across an internal Docker bridge network, logically divided into five functional layers:

- **Layer 1 — Persistence & Caching Tier (PostgreSQL 15 & Redis 7):** PostgreSQL acts as the authoritative source of truth holding 50,000 enterprise hardware records seeded automatically on first boot via init.sql. Redis operates strictly as a bounded 256 MB volatile cache with an allkeys-lru eviction policy, ensuring predictable memory consumption on the host machine.
- **Layer 2 — Asynchronous Connection Pool & Lifespan Manager:** Managed by FastAPI's @asynccontextmanager lifespan function, the application negotiates 10 to 30 persistent binary TCP connections to PostgreSQL via asyncpg and initializes an asynchronous Redis client pool before yielding control to the Uvicorn server.
- **Layer 3 — In-Process 5-Stage ASGI Middleware Engine:** Intercepts every incoming HTTP request before route execution. Stage 1 filters non-GET methods and bypass routes (/nocache, /telemetry, /auth, /docs). Stage 2 verifies HMAC-SHA256 JWT signatures and RBAC roles in CPU memory for protected paths (/admin/analytics), blocking invalid tokens in ~0.16 ms. Stage 3 constructs a tenant-scoped cache key. Stage 4 executes a fast-path O(1) Redis GET; if present, it returns immediately with X-Cache: HIT. Stage 5 handles misses by acquiring a key-specific asyncio.Lock, performing a double-checked Redis read (returning X-Cache: HIT-COALESCED for waiting coroutines), or querying PostgreSQL once (X-Cache: MISS) and writing the JSON payload to Redis with a 60-second TTL.
- **Layer 4 — Real-Time Telemetry & Load Generation Tier:** A Streamlit frontend on port 8501 polls the middleware's lock-free telemetry endpoints every 2 seconds to visualize hit ratios, saved DB queries, latency comparisons, a 40-request circular ring buffer, an interactive 50-request stampede simulator, and a JWT/RBAC inspector. Simultaneously, Locust on port 8089 drives continuous weighted traffic across hot keys, full catalog sweeps, uncached baselines, and admin routes.
- **Layer 5 — Container Orchestration & Healthcheck Sequencing:** Defined in docker-compose.yml, this layer enforces deterministic startup ordering. Using pg_isready and redis-cli ping healthchecks paired with depends_on: condition: service_healthy, Docker prevents the FastAPI container from booting until both backing stores are actively accepting connections.

5.3 Request Execution Pipeline Matrix

Pipeline Scenario	Trigger Condition	Middleware Action & Lock State	X-Cache Header	Typical Latency
Pre-Cache Auth Block	Tampered/Missing JWT on /admin/*	Rejected in CPU memory before Redis/DB	AUTH-BLOCKED (401)	~0.16 ms
Pre-Cache RBAC Block	Valid JWT with role != admin	Rejected in CPU memory before Redis/DB	RBAC-BLOCKED (403)	~0.21 ms
Fast-Path Cache Hit	Warm key found on initial Redis GET	Immediate JSON response from Redis RAM	HIT (200)	~1.28 – 16 ms
Coalesced Stampede Hit	Cold key; coroutine waited on asyncio.Lock	Wakes after leader write; hits on 2nd Redis GET	HIT-COALESCED (200)	~45 – 54 ms
True Cold Cache Miss	Cold key; leader acquires asyncio.Lock	Queries PostgreSQL, serializes JSON, SETEX 60s	MISS (200)	~52 – 328 ms
Uncached / Write Bypass	GET /nocache/* or PUT /products/*	Skips cache lookup; executes DB SQL + DEL key	BYPASS (200)	~55 – 97 ms

6. SCHEMA

6.1 Single-Entity High-Cardinality Schema Justification

Because the primary objective of this system is to evaluate in-process ASGI caching throughput, single-flight lock coalescing, and cache eviction mechanics, the relational schema is intentionally modeled as a flattened, high-cardinality single-entity relation named products containing 50,000 synthetic enterprise hardware records. Eliminating multi-table foreign-key joins ensures that uncached baseline measurements reflect pure B-tree index scans, heap fetches, and JSONB serialization overhead in PostgreSQL without confounding join-planner variability.

6.2 PostgreSQL DDL & Automated 50,000-Row Seeding Code (postgres/init.sql)

File: postgres/init.sql — Schema Creation, Indexing, and 50,000-Row Synthetic Enterprise Catalog Seeding

```
CREATE TABLE IF NOT EXISTS products (
    id SERIAL PRIMARY KEY,
    sku VARCHAR(64) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    category VARCHAR(100) NOT NULL,
    price NUMERIC(10, 2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    specs JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_products_category ON products(category);
CREATE INDEX idx_products_stock ON products(stock);

-- Automated deterministic seeding of 50,000 enterprise component records
INSERT INTO products (sku, name, category, price, stock, specs)
SELECT
    UPPER(SUBSTRING(MD5(i::TEXT) FROM 1 FOR 12)),
    'Enterprise_Component_' || i,
    (ARRAY['Processors','Memory_Modules','NVMe_Storage','Network_Cards',
           'Power_Supplies','Motherboards','Cooling_Units','Chassis_Racks'])[(i % 8) + 1],
    ROUND(((RANDOM() * 1950) + 50)::NUMERIC, 2),
    FLOOR(RANDOM() * 500)::INT,
    jsonb_build_object(
        'batch_id', (i % 100) + 1,
        'certified', (i % 2 = 0),
        'rack_units', (i % 4) + 1,
        'warranty_years', (i % 5) + 1
    )
FROM generate_series(1, 50000) AS s(i)
ON CONFLICT DO NOTHING;
```

6.3 Schema Data Dictionary

Column Name	SQL Data Type	Constraints	Index Type	System Role & Description
id	SERIAL (INT4)	PRIMARY KEY	B-Tree (Unique)	Auto-incrementing identifier (1 to 50,000) mapped to /products/{id}.
sku	VARCHAR(64)	UNIQUE, NOT NULL	B-Tree (Unique)	12-character hexadecimal stock-keeping identifier generated via MD5.
name	VARCHAR(255)	NOT NULL	None	Enterprise component label (e.g., Enterprise_Component_20).
category	VARCHAR(100)	NOT NULL	B-Tree (idx_products_category)	Hardware classification across 8 distinct categories; used in analytics.
price	NUMERIC(10, 2)	NOT NULL	None	Component unit price ($50.00–$2,000.00); aggregated via AVG(price).
stock	INT4	NOT NULL, DEFAULT 0	B-Tree (idx_products_stock)	Mutable warehouse quantity targeted by PUT /products/{id} invalidations.
specs	JSONB	NOT NULL	None	Binary JSON document storing batch_id, certified, rack_units, warranty_years.
created_at	TIMESTAMPTZ	DEFAULT NOW()	None	Audit timestamp recording initial container initialization.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	None	Audit timestamp updated atomically during PUT stock mutations.

6.4 Relational-to-Redis Key Namespacing Schema

Access Scope	Route Pattern	Synthesized Redis Key Format	TTL & Eviction Policy
Public Catalog Read	GET /products/{product_id}	GET:/products/{product_id}:public	60s TTL | allkeys-lru (256 MB cap)
Tenant-Scoped Admin	GET /admin/analytics	GET:/admin/analytics:role:{role}:user:{sub}	60s TTL | Isolated per JWT sub & role
Uncached Control	GET /nocache/products/{id}	None (Bypasses Redis completely)	N/A (Direct PostgreSQL query)
Stock Mutation	PUT /products/{product_id}	Issues explicit DEL GET:/products/{id}:public	Immediate eviction on DB commit

7. IMPLEMENTATION DETAILS: DBMS, APPLICATION TECHNOLOGIES, AND DATABASE CONNECTIVITY

7.1 Layer 5 Container Orchestration Code (docker-compose.yml)

File: docker-compose.yml — 5-Service Microservice Stack with Healthcheck Sequencing

```
services:
  postgres:
    image: postgres:15-alpine
    container_name: proxy_postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: proxy_bench
    ports:
      - "5432:5432"
    volumes:
      - ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
      - pg_bench_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d proxy_bench"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: proxy_redis
    ports:
      - "6379:6379"
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  middleware:
    build: ./middleware
    container_name: proxy_middleware
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://postgres:postgres@postgres:5432/proxy_bench
      REDIS_URL: redis://redis:6379/0
      JWT_SECRET: vit-cse-hmac-sha256-secret-key-2026
      CACHE_TTL: "60"
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy

  frontend:
    build: ./frontend
    container_name: proxy_frontend
    ports:
```

```
      - "8501:8501"
    environment:
      MIDDLEWARE_URL: http://middleware:8000
    depends_on:
      - middleware

  locust:
    build: ./locust
    container_name: proxy_locust
    ports:
      - "8089:8089"
    depends_on:
      - middleware

volumes:
  pg_bench_data:
```

7.2 DBMS Connectivity, Docker Healthchecks vs. FastAPI Lifespan & os.getenv

- **1. Infrastructure Readiness (Docker pg_isready & redis-cli ping):** When docker compose up runs, Docker starts containers concurrently. PostgreSQL requires several seconds to allocate shared memory and seed 50,000 rows via init.sql. The healthcheck continuously runs pg_isready -U postgres -d proxy_bench inside the Postgres container and redis-cli ping inside Redis. Pairing this with depends_on: condition: service_healthy prevents the middleware container from booting until both databases confirm they are accepting TCP connections.
- **2. Application Resource Management (FastAPI lifespan Context Manager):** Once Docker starts the middleware container, the @asynccontextmanager lifespan function executes before Uvicorn accepts HTTP traffic. Rather than merely checking if the database is awake, it negotiates and opens 10 to 30 persistent binary connections (asyncpg.create_pool) and initializes the asynchronous Redis pool. Only after these connection pools are established does lifespan execute yield to start the ASGI event loop.
- **3. Top-Level Environment Injection (os.getenv & DATABASE_URL):** DATABASE_URL packages the routing and credential parameters into a single URI (postgresql://postgres:postgres@postgres:5432/proxy_bench). Executing os.getenv() at the top of main.py right after module imports reads the container's OS environment once at startup and binds global constants (DATABASE_URL, REDIS_URL, JWT_SECRET, CACHE_TTL) in memory for all route handlers and middleware coroutines.

7.3 FastAPI In-Process ASGI Middleware Source Code (middleware/main.py)

File: middleware/main.py — Lifespan Pool, 5-Stage ASGI Middleware, RBAC, Cache-Aside & Telemetry Routes

```
import os, time, json, asyncio
from collections import deque
from contextlib import asynccontextmanager
from pydantic import BaseModel
import asyncpg, redis.asyncio as redis, jwt
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import Response

DATABASE_URL = os.getenv("DATABASE_URL",
"postgresql://postgres:postgres@postgres:5432/proxy_bench")
REDIS_URL    = os.getenv("REDIS_URL", "redis://redis:6379/0")
JWT_SECRET   = os.getenv("JWT_SECRET", "vit-cse-hmac-sha256-secret-key-2026")
CACHE_TTL    = int(os.getenv("CACHE_TTL", "60"))

db_pool: asyncpg.Pool = None
redis_client: redis.Redis = None
key_locks: dict[str, asyncio.Lock] = {}

metrics = {
    "total_requests": 0, "cache_hits": 0, "coalesced_hits": 0,
    "cache_misses": 0, "bypass_requests": 0, "auth_blocked": 0,
    "cached_latency_sum": 0.0, "cached_count": 0,
    "direct_latency_sum": 0.0, "direct_count": 0,
}
request_stream = deque(maxlen=40)

def record_telemetry(method: str, path: str, status: str, latency: float, key: str, code: int):
    metrics["total_requests"] += 1
    if status == "HIT":
        metrics["cache_hits"] += 1
        metrics["cached_latency_sum"] += latency
        metrics["cached_count"] += 1
    elif status == "HIT-COALESCED":
        metrics["coalesced_hits"] += 1
        metrics["cached_latency_sum"] += latency
        metrics["cached_count"] += 1
    elif status == "MISS":
        metrics["cache_misses"] += 1
        metrics["cached_latency_sum"] += latency
        metrics["cached_count"] += 1
    elif status == "BYPASS":
        metrics["bypass_requests"] += 1
        if path.startswith("/nocache"):
            metrics["direct_latency_sum"] += latency
            metrics["direct_count"] += 1
    elif status in ("AUTH-BLOCKED", "RBAC-BLOCKED"):
        metrics["auth_blocked"] += 1
    request_stream.appendleft({
        "time": time.strftime("%H:%M:%S"), "method": method, "path": path,
        "cache_status": status, "latency_ms": latency, "cache_key": key, "status_code": code
    })

@asynccontextmanager
async def lifespan(app: FastAPI):
    global db_pool, redis_client
```

```
    db_pool = await asyncpg.create_pool(dsn=DATABASE_URL, min_size=10, max_size=30)
    redis_client = redis.from_url(REDIS_URL, decode_responses=True)
    yield
    await redis_client.close()
    await db_pool.close()

app = FastAPI(title="In-Process ASGI Caching Middleware", lifespan=lifespan)

@app.middleware("http")
async def asgi_caching_stampede_middleware(request: Request, call_next):
    t_start = time.perf_counter()
    path, method = request.url.path, request.method

    if method != "GET" or path.startswith(("/nocache", "/telemetry", "/auth", "/docs",
"/openapi.json")):
        response = await call_next(request)
        latency = round((time.perf_counter() - t_start) * 1000, 2)
        response.headers["X-Cache"] = "BYPASS"
        response.headers["X-Process-Time-Ms"] = str(latency)
        if not path.startswith(("/telemetry", "/docs", "/openapi.json")):
            record_telemetry(method, path, "BYPASS", latency, "None", response.status_code)
        return response

    cache_scope = "public"
    if path.startswith("/admin"):
        auth_hdr = request.headers.get("Authorization", "")
        if not auth_hdr.startswith("Bearer "):
            latency = round((time.perf_counter() - t_start) * 1000, 2)
            record_telemetry(method, path, "AUTH-BLOCKED", latency, "None", 401)
            return Response(json.dumps({"detail": "Missing Bearer token"}), status_code=401,
                            media_type="application/json",
                            headers={"X-Cache": "AUTH-BLOCKED", "X-Cache-Key": "None", "X-
Process-Time-Ms": str(latency)})
        token = auth_hdr.split(" ", 1)[1]
        try:
            payload = jwt.decode(token, JWT_SECRET, algorithms=["HS256"])
        except jwt.PyJWTError:
            latency = round((time.perf_counter() - t_start) * 1000, 2)
            record_telemetry(method, path, "AUTH-BLOCKED", latency, "None", 401)
            return Response(json.dumps({"detail": "Invalid HMAC-SHA256 signature"}),
status_code=401,
                            media_type="application/json",
                            headers={"X-Cache": "AUTH-BLOCKED", "X-Cache-Key": "None", "X-
Process-Time-Ms": str(latency)})
        if payload.get("role") != "admin":
            latency = round((time.perf_counter() - t_start) * 1000, 2)
            record_telemetry(method, path, "RBAC-BLOCKED", latency, "None", 403)
            return Response(json.dumps({"detail": "Forbidden: Admin role required"}),
status_code=403,
                            media_type="application/json",
                            headers={"X-Cache": "RBAC-BLOCKED", "X-Cache-Key": "None", "X-
Process-Time-Ms": str(latency)})
        cache_scope = f"role:{payload['role']}:user:{payload['sub']}"

    cache_key = f"{method}:{path}:{cache_scope}"

    cached_payload = await redis_client.get(cache_key)
    if cached_payload is not None:
        latency = round((time.perf_counter() - t_start) * 1000, 2)
        record_telemetry(method, path, "HIT", latency, cache_key, 200)
```

```
        return Response(content=cached_payload, media_type="application/json",
                        headers={"X-Cache": "HIT", "X-Cache-Key": cache_key,
                                 "X-Process-Time-Ms": str(latency), "Cache-Control":
f"public,max-age={CACHE_TTL}"})

    lock = key_locks.setdefault(cache_key, asyncio.Lock())
    async with lock:
        coalesced_payload = await redis_client.get(cache_key)
        if coalesced_payload is not None:
            latency = round((time.perf_counter() - t_start) * 1000, 2)
            record_telemetry(method, path, "HIT-COALESCED", latency, cache_key, 200)
            return Response(content=coalesced_payload, media_type="application/json",
                            headers={"X-Cache": "HIT-COALESCED", "X-Cache-Key": cache_key, "X-
Process-Time-Ms": str(latency)})

        response = await call_next(request)
        body_bytes = b"".join([chunk async for chunk in response.body_iterator])
        if response.status_code == 200:
            await redis_client.setex(cache_key, CACHE_TTL, body_bytes.decode("utf-8"))
        latency = round((time.perf_counter() - t_start) * 1000, 2)
        record_telemetry(method, path, "MISS", latency, cache_key, response.status_code)
        return Response(content=body_bytes, status_code=response.status_code,
media_type="application/json",
                        headers={"X-Cache": "MISS", "X-Cache-Key": cache_key, "X-Process-Time-
Ms": str(latency)})

class TokenReq(BaseModel):
    user_id: int
    role: str = "admin"

class StockUpdateReq(BaseModel):
    stock: int

@app.post("/auth/token")
async def issue_token(req: TokenReq):
    token = jwt.encode({"sub": str(req.user_id), "role": req.role}, JWT_SECRET,
algorithm="HS256")
    return {"access_token": token, "role": req.role, "user_id": req.user_id}

@app.get("/products/{product_id}")
async def get_product_cached(product_id: int):
    async with db_pool.acquire() as conn:
        row = await conn.fetchrow("SELECT id, sku, name, category, price, stock, specs FROM
products WHERE id = $1", product_id)
        if not row:
            raise HTTPException(status_code=404, detail="Product not found")
        return {**dict(row), "price": float(row["price"]), "specs": json.loads(row["specs"]),
"served_by": "PostgreSQL Origin"}

@app.get("/nocache/products/{product_id}")
async def get_product_nocache(product_id: int):
    async with db_pool.acquire() as conn:
        row = await conn.fetchrow("SELECT id, sku, name, category, price, stock, specs FROM
products WHERE id = $1", product_id)
        if not row:
            raise HTTPException(status_code=404, detail="Product not found")
        return {**dict(row), "price": float(row["price"]), "specs": json.loads(row["specs"]),
"served_by": "Direct PostgreSQL (/nocache)"}

@app.put("/products/{product_id}")
```

```
async def update_product_stock(product_id: int, req: StockUpdateReq):
    async with db_pool.acquire() as conn:
        await conn.execute("UPDATE products SET stock = $1, updated_at = NOW() WHERE id = $2",
req.stock, product_id)
    cache_key = f"GET:/products/{product_id}:public"
    await redis_client.delete(cache_key)
    return {"status": "updated", "invalidated_key": cache_key, "new_stock": req.stock}

@app.get("/admin/analytics")
async def get_admin_analytics():
    async with db_pool.acquire() as conn:
        row = await conn.fetchrow(
            "SELECT COUNT(*) AS total_records, SUM(stock) AS total_inventory_units, "
            "ROUND(AVG(price), 2) AS avg_unit_price, COUNT(DISTINCT category) AS
distinct_categories FROM products"
        )
        return {
            "total_records": row["total_records"],
            "total_inventory_units": row["total_inventory_units"],
            "avg_unit_price": float(row["avg_unit_price"]),
            "distinct_categories": row["distinct_categories"],
            "generated_at": time.strftime("%H:%M:%S")
        }

@app.get("/telemetry/metrics")
async def get_telemetry():
    eligible = metrics["cache_hits"] + metrics["coalesced_hits"] + metrics["cache_misses"]
    hit_ratio = round(((metrics["cache_hits"] + metrics["coalesced_hits"]) / eligible * 100), 2)
if eligible > 0 else 0.0
    avg_cached = round(metrics["cached_latency_sum"] / metrics["cached_count"], 2) if
metrics["cached_count"] > 0 else 0.0
    avg_direct = round(metrics["direct_latency_sum"] / metrics["direct_count"], 2) if
metrics["direct_count"] > 0 else 0.0
    return {
        **metrics, "cache_hit_ratio": hit_ratio,
        "db_queries_saved": metrics["cache_hits"] + metrics["coalesced_hits"],
        "avg_cached_latency_ms": avg_cached, "avg_direct_latency_ms": avg_direct,
        "recent_stream": list(request_stream)
    }

@app.post("/telemetry/reset")
async def reset_telemetry():
    await redis_client.flushdb()
    for k in metrics:
        metrics[k] = 0 if isinstance(metrics[k], int) else 0.0
    request_stream.clear()
    return {"status": "reset"}
```

7.4 Streamlit Real-Time Telemetry & Stampede Simulator Code (frontend/dashboard.py)

File: frontend/dashboard.py — Live KPI Dashboard, 50-Request Burst Simulator & JWT/RBAC Inspector

```
import os, time, asyncio, httpx, jwt
import pandas as pd
import plotly.express as px
import streamlit as st

MIDDLEWARE_URL = os.getenv("MIDDLEWARE_URL", "http://middleware:8000")
JWT_SECRET     = os.getenv("JWT_SECRET", "vit-cse-hmac-sha256-secret-key-2026")

st.set_page_config(page_title="ASGI Caching Middleware", layout="wide")
st.title("In-Process ASGI Caching Middleware & Stampede Mitigator")
st.caption("Real-Time Telemetry: Single-Flight (asyncio.Lock) Stampede Prevention • HMAC-SHA256
JWT Namespacing • 50,000 PostgreSQL Records")

with st.sidebar:
    st.subheader("Control Panel")
    auto_refresh = st.toggle("Auto-Refresh Live Telemetry (2s)", value=True)
    if st.button("Reset Redis Cache & Metrics", use_container_width=True):
        httpx.post(f"{MIDDLEWARE_URL}/telemetry/reset", timeout=5.0)
        st.rerun()

data = httpx.get(f"{MIDDLEWARE_URL}/telemetry/metrics", timeout=5.0).json()

c1, c2, c3, c4, c5, c6 = st.columns(6)
c1.metric("Total Requests", f"{data['total_requests']:,}")
c2.metric("Cache Hit Ratio", f"{data['cache_hit_ratio']}%")
c3.metric("Stampedes Coalesced", f"{data['coalesced_hits']:,}")
c4.metric("DB Queries Saved", f"{data['db_queries_saved']:,}")
c5.metric("Avg Cached Latency", f"{data['avg_cached_latency_ms']} ms")
c6.metric("Avg Direct DB Latency", f"{data['avg_direct_latency_ms']} ms")

tab1, tab2, tab3 = st.tabs([
    "Live Benchmark & Stampede Telemetry",
    "Interactive Stampede Burst Simulator",
    "JWT Auth & RBAC Key Namespacing Inspector"
])

with tab1:
    col_l, col_r = st.columns(2)
    with col_l:
        st.subheader("Traffic Disposition Breakdown")
        df_disp = pd.DataFrame({
            "Disposition": [
                "Fast-Path Cache Hit (HIT)", "Stampede Coalesced (HIT-COALESCED)",
                "True Cache Miss -> DB (MISS)", "Uncached Baseline (BYPASS)", "Pre-Cache Auth
Blocked (401/403)"
            ],
            "Count": [
                data["cache_hits"], data["coalesced_hits"],
                data["cache_misses"], data["bypass_requests"], data["auth_blocked"]
            ]
        })
        st.plotly_chart(px.bar(df_disp, x="Count", y="Disposition", orientation="h",
text="Count"), use_container_width=True)
```

```
    with col_r:
        st.subheader("Latency Comparison: Middleware vs Direct PostgreSQL")
        df_lat = pd.DataFrame({
            "Path Architecture": ["In-Process Caching Middleware", "Direct PostgreSQL
(/nocache)"],
            "Avg Latency (ms)": [data["avg_cached_latency_ms"], data["avg_direct_latency_ms"]]
        })
        st.plotly_chart(px.bar(df_lat, x="Path Architecture", y="Avg Latency (ms)", text="Avg
Latency (ms)"), use_container_width=True)

    st.subheader("Live Request Stream (Last 40 Intercepted Requests)")
    if data["recent_stream"]:
        st.dataframe(pd.DataFrame(data["recent_stream"]), use_container_width=True,
hide_index=True)

with tab2:
    st.subheader("Prove Single-Flight (asyncio.Lock) Stampede Mitigation")
    burst_n = st.slider("Concurrent Burst Requests", 10, 100, 50)
    target_id = st.number_input("Target Product ID (1 - 50,000)", min_value=1, max_value=50000,
value=6283)
    if st.button("Fire Concurrent Stampede Burst"):
        async def run_burst(n: int, pid: int):
            async with httpx.AsyncClient(base_url=MIDDLEWARE_URL, timeout=15.0) as client:
                tasks = [client.get(f"/products/{pid}") for _ in range(n)]
                resps = await asyncio.gather(*tasks)
                return [{
                    "Request #": i + 1, "HTTP Status": r.status_code,
                    "X-Cache Header": r.headers.get("x-cache"),
                    "X-Cache-Key": r.headers.get("x-cache-key"),
                    "Latency (ms)": float(r.headers.get("x-process-time-ms", 0.0))
                } for i, r in enumerate(resps)]
        rows = asyncio.run(run_burst(burst_n, target_id))
        df_b = pd.DataFrame(rows)
        misses = (df_b["X-Cache Header"] == "MISS").sum()
        coal   = (df_b["X-Cache Header"] == "HIT-COALESCED").sum()
        hits   = (df_b["X-Cache Header"] == "HIT").sum()
        st.success(f"Burst Complete for Product #{target_id}! DB Queries Executed (MISS):
{misses} | Coalesced Stampede Hits (HIT-COALESCED): {coal} | Standard Hits (HIT): {hits}")
        st.dataframe(df_b, use_container_width=True, hide_index=True)

with tab3:
    st.subheader("Stateless JWT Authentication & Role-Scoped Cache Keys")
    uid = st.number_input("User ID (sub claim)", min_value=1, max_value=999, value=42)
    role_opt = st.selectbox("User Role (role claim)", ["admin", "user", "tampered_signature"])
    if st.button("Send Request to GET /admin/analytics"):
        secret = JWT_SECRET if role_opt != "tampered_signature" else "wrong-tampered-secret-key"
        token = jwt.encode({"sub": str(uid), "role": role_opt}, secret, algorithm="HS256")
        r = httpx.get(f"{MIDDLEWARE_URL}/admin/analytics", headers={"Authorization": f"Bearer
{token}"}, timeout=10.0)
        st.code(
            f"HTTP Status: {r.status_code}
"
            f"X-Cache: {r.headers.get('x-cache')}
"
            f"X-Cache-Key: {r.headers.get('x-cache-key')} {'(Rejected Before Redis)' if
r.headers.get('x-cache-key') == 'None' else ''}
"
            f"X-Process-Time-Ms: {r.headers.get('x-process-time-ms')} ms"
        )
        st.json(r.json())
```

7.5 Locust Concurrency Workload & Benchmark Code (locust/locustfile.py)

File: locust/locustfile.py — Weighted Multi-Scenario Coroutine Swarm (6:2:2:1 Ratio)

```
import random
from locust import HttpUser, task, between

class CachingProxyBenchmarkUser(HttpUser):
    wait_time = between(0.01, 0.05)

    def on_start(self):
        """Issue an admin JWT token once when each virtual user spawns."""
        uid = random.randint(1, 25)
        resp = self.client.post(
            "/auth/token",
            json={"user_id": uid, "role": "admin"},
            name="/auth/token (Setup)"
        )
        if resp.status_code == 200:
            self.token = resp.json().get("access_token", "")
        else:
            self.token = ""

    @task(6)
    def get_hot_cached_products(self):
        """Weight 6 (~54.5%): Repeatedly query hot product IDs (1-30) resident in Redis RAM."""
        pid = random.randint(1, 30)
        self.client.get(
            f"/products/{pid}",
            name="1. [CACHED] Hot Products (IDs 1-30)"
        )

    @task(2)
    def get_cold_catalog_sweep(self):
        """Weight 2 (~18.2%): Sweep randomly across all 50,000 products to trigger misses & LRU
eviction."""
        pid = random.randint(31, 50000)
        self.client.get(
            f"/products/{pid}",
            name="2. [CACHED] 50k Catalog Sweep"
        )

    @task(2)
    def get_uncached_baseline(self):
        """Weight 2 (~18.2%): Bypass Redis to benchmark raw PostgreSQL read latency."""
        pid = random.randint(1, 50000)
        self.client.get(
            f"/nocache/products/{pid}",
            name="3. [DIRECT DB] Uncached Baseline"
        )
```

```
    @task(1)
     def get_authenticated_admin_analytics(self):
        """Weight 1 (~9.1%): Exercise HMAC-SHA256 verification and tenant-scoped Redis caching."""
        if self.token:
            self.client.get(
                "/admin/analytics",
                headers={"Authorization": f"Bearer {self.token}"},
                name="4. [AUTH ADMIN] Scoped Cache"
            )
```

8. IMPLEMENTED FEATURES

8.1 Functional Capabilities Overview

- **1. Zero-Hop Stateless HMAC-SHA256 JWT Gatekeeping:** Validates cryptographic token signatures in CPU memory using PyJWT before any Redis or PostgreSQL socket operations occur. Corrupted tokens are rejected in 0.16 ms (HTTP 401, X-Cache: AUTH-BLOCKED), and insufficient roles are rejected in 0.21 ms (HTTP 403, X-Cache: RBAC-BLOCKED).
- **2. Dynamic Role- and Tenant-Scoped Key Namespacing:** Automatically embeds JWT sub and role claims into the Redis key structure (GET:/admin/analytics:role:admin:user:42), guaranteeing strict multi-tenant cache isolation.
- **3. In-Process Single-Flight Stampede Mitigator (asyncio.Lock):** Synchronizes concurrent coroutines requesting the same cold cache key within Uvicorn's single event loop. Exactly one leader queries PostgreSQL while waiting followers execute a double-checked Redis read upon lock release (X-Cache: HIT-COALESCED).
- **4. Atomic Cache-Aside Stock Mutation & Eviction (PUT /products/{id}):** Updates product inventory in PostgreSQL and immediately issues an explicit Redis DEL GET:/products/{id}:public command to evict stale cache entries.
- **5. Uncached Control Baseline Route (GET /nocache/products/{id}):** Intentionally bypasses the caching middleware to execute direct parameterized SQL lookups against PostgreSQL, providing a real-time empirical baseline for calculating cache speedup ratios.
- **6. Real-Time Streamlit Telemetry & Stampede Burst Simulator:** Provides a 3-tab operational command center featuring a 6-card KPI scoreboard, traffic disposition bar charts, latency comparison histograms, a 40-request live circular ring buffer, an interactive 50-request asynchronous burst generator, and a live JWT/RBAC inspector.
- **7. Weighted Multi-Scenario Locust Stress Engine:** Simulates realistic skewed production traffic with task weights 6:2:2:1 across Hot Products (IDs 1–30), 50k Catalog Sweeps, Uncached Baselines, and Authenticated Admin Analytics.

9. TESTING, RESULTS, SCREENSHOTS, AND DISCUSSION

9.1 Locust Load Testing Configuration & Aggregate Benchmark Results

Load testing was conducted using Locust (http://localhost:8089) targeting http://middleware:8000 with a peak concurrency of 10 virtual users ramped up at 5 users/second. The workload distributed requests across four weighted tasks (Total Weight = 11): Weight 6 (~54.5%) for Hot Products (IDs 1–30), Weight 2 (~18.2%) for 50k Catalog Sweep, Weight 2 (~18.2%) for Direct DB Uncached Baseline, and Weight 1 (~9.1%) for Auth Admin Scoped Cache.

HTTP Type	Endpoint / Task Name	Weight	Requests	Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Avg (ms)	RPS
POST	/auth/token (Setup)	Setup	20	0 (0%)	52	200	200	81.48	0.0
GET	1. [CACHED] Hot Products (IDs 1-30)	6 (54.5%)	2,889	0 (0%)	25	99	170	41.82	58.0
GET	2. [CACHED] 50k Catalog Sweep	2 (18.2%)	925	0 (0%)	99	230	380	144.20	17.9
GET	3. [DIRECT DB] Uncached Baseline	2 (18.2%)	943	0 (0%)	85	210	300	121.51	19.3
GET	4. [AUTH ADMIN] Scoped Cache	1 (9.1%)	482	0 (0%)	25	120	190	36.99	10.5
ALL	Aggregated System Total	11 (100%)	5,259	0 (0%)	48	170	260	73.83	105.7

9.2 Longitudinal Streamlit Telemetry Progression

As continuous Locust traffic and manual burst tests progressed across the evaluation session, the Streamlit telemetry scoreboard captured the steady rise in cache efficiency and cumulative database queries saved:

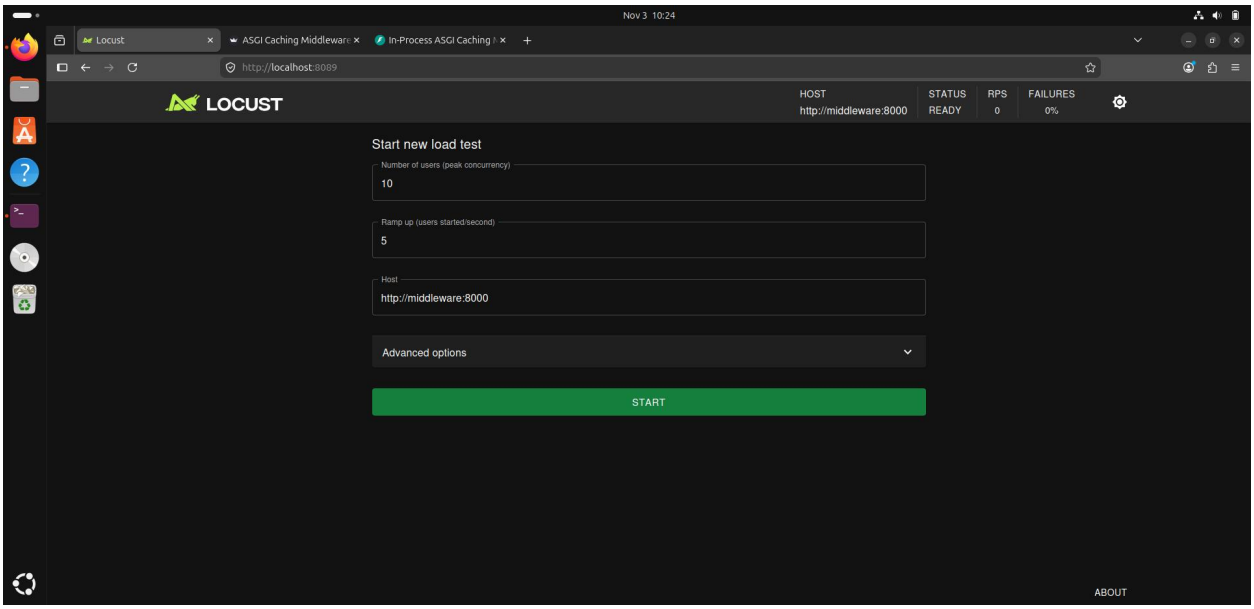
Evaluation Checkpoint (Screenshot Ref)	Total Requests	Cache Hit Ratio	Stampedes Coalesced	DB Queries Saved	Avg Cached Latency	Avg Direct DB Latency	Speedup
Early Load Phase	1,274	73.59%	2	770	31.12 ms	73.07 ms	2.35x
Admin Cold Miss Phase	19,415	77.65%	43	12,356	23.87 ms	61.03 ms	2.56x
Admin Warm Hit Phase	21,443	77.62%	43	13,661	16.86 ms	54.38 ms	3.23x
Tenant Isolation Phase	24,741	77.70%	43	15,771	14.08 ms	54.55 ms	3.87x
RBAC Verification Phase	28,075	77.73%	45	17,885	14.37 ms	53.37 ms	3.71x
Auth Security Phase	30,447	77.75%	45	19,401	13.33 ms	55.22 ms	4.14x

9.3 Single-Flight Stampede Simulator & Security Inspector Results

Test Case & Screenshot Reference	Input Parameters	Observed HTTP & X-Cache Status	Computed Redis Cache Key	Measured Latency	Empirical Conclusion
50-Request Cold Stampede Burst	50 concurrent GETs on cold Product #6283	1 MISS | 20 HIT-COALESCED | 29 HIT (All HTTP 200)	GET:/products/6283:public	52.01 ms (MISS) | 45.17–54.10 ms (COALESCED)	98% DB query reduction (49/50 queries prevented)
Admin Cold Cache Miss	user_id: 42, role: admin -> GET /admin/analytics	HTTP 200 | X-Cache: MISS	GET:/admin/analytics:role:admin:user:42	328.38 ms	Full 50k-row SQL aggregation executed & cached
Admin Warm Fast-Path Hit	user_id: 42, role: admin -> GET /admin/analytics	HTTP 200 | X-Cache: HIT	GET:/admin/analytics:role:admin:user:42	1.28 ms	256.5x speedup via in-RAM Redis retrieval
Multi-Tenant Key Isolation	user_id: 41, role: admin -> GET /admin/analytics	HTTP 200 | X-Cache: MISS	GET:/admin/analytics:role:admin:user:41	214.56 ms	Zero cross-tenant data leakage between admins
Pre-Cache RBAC Rejection	user_id: 41, role: user -> GET /admin/analytics	HTTP 403 | X-Cache: RBAC-BLOCKED	None (Rejected Before Redis)	0.21 ms	Unauthorized role blocked in CPU prior to Redis/DB
Tampered JWT Signature Block	user_id: 41, role: tampered_signature	HTTP 401 | X-Cache: AUTH-BLOCKED	None (Rejected Before Redis)	0.16 ms	Invalid HMAC-SHA256 digest dropped in 0.16 ms

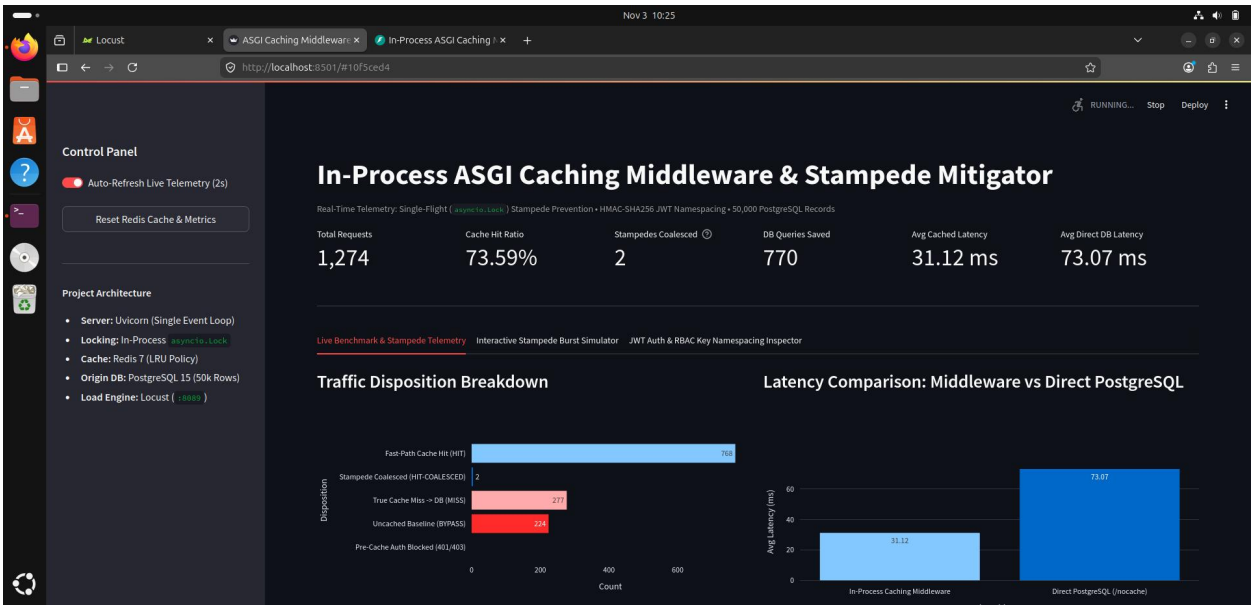
9.4 Discussion of Empirical Verification Screenshots (1.png – 19.png)

- **Locust Load Test Initialization:**



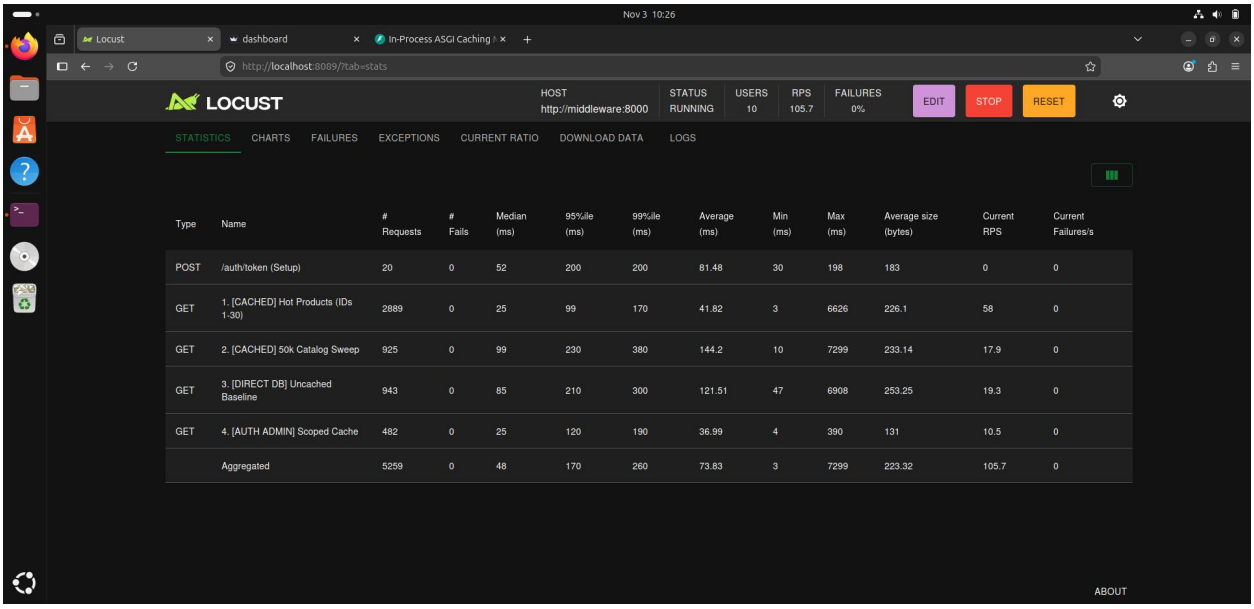
Configures Locust (http://localhost:8089) with 10 peak concurrent users, a ramp-up rate of 5 users/second, and target host http://middleware:8000 over Docker internal DNS. Cluster status shows READY with 0% initial failures.

- **Streamlit Live Benchmark & Telemetry Overview:**



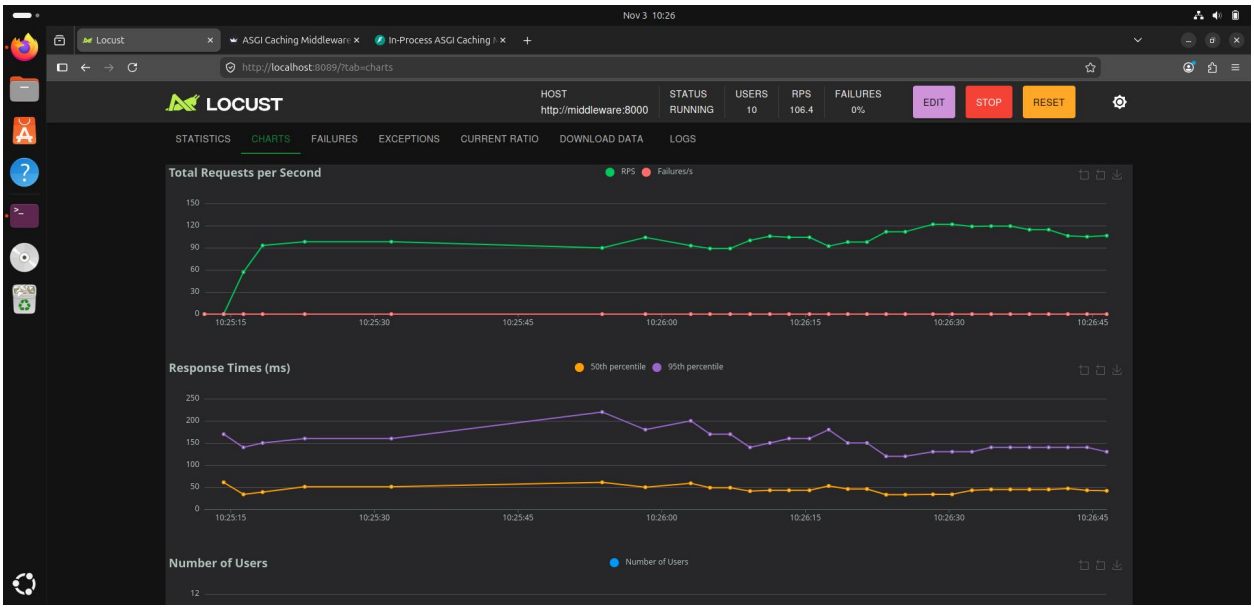
Displays the 6-card KPI row after 1,274 requests: 73.59% Cache Hit Ratio, 2 Stampedes Coalesced, 770 DB Queries Saved (768 Fast-Path Hits + 2 Coalesced Hits), 277 True Cache Misses, and 224 Uncached Baselines, achieving a 2.35x speedup (31.12 ms cached vs 73.07 ms direct PostgreSQL).

- **Locust Aggregate Endpoint Statistics Table:**



Documents 5,259 total requests with 0% failures at 105.7 RPS. Hot cached products (IDs 1–30) achieved a 25 ms median latency (3 ms min) at 58 RPS, while the 50k catalog sweep recorded a 99 ms median and the uncached DB baseline recorded an 85 ms median (121.51 ms mean).

- **Locust Real-Time Throughput & Latency Percentile Charts:**



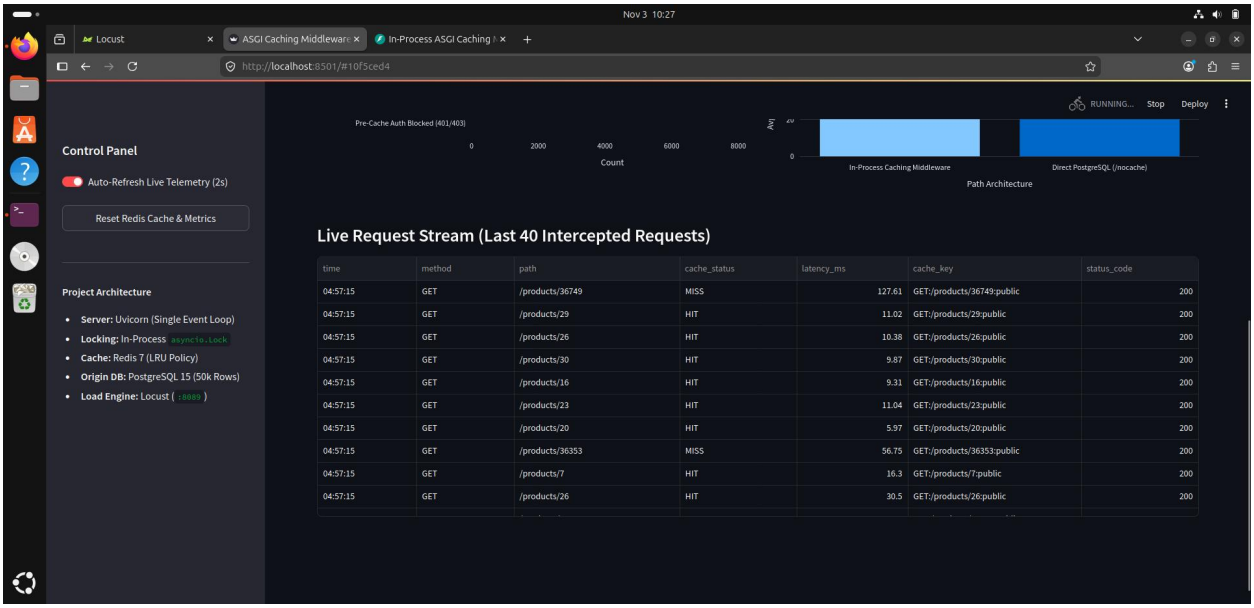
Time-series telemetry showing total throughput climbing during the 2-second ramp-up and stabilizing at 100–120 RPS with 0 failures/s. The 50th percentile response time remains steady at 35–45 ms, while the 95th percentile plateaus safely below 180 ms.

- **Locust Concurrency Progression & Worker Stability:**



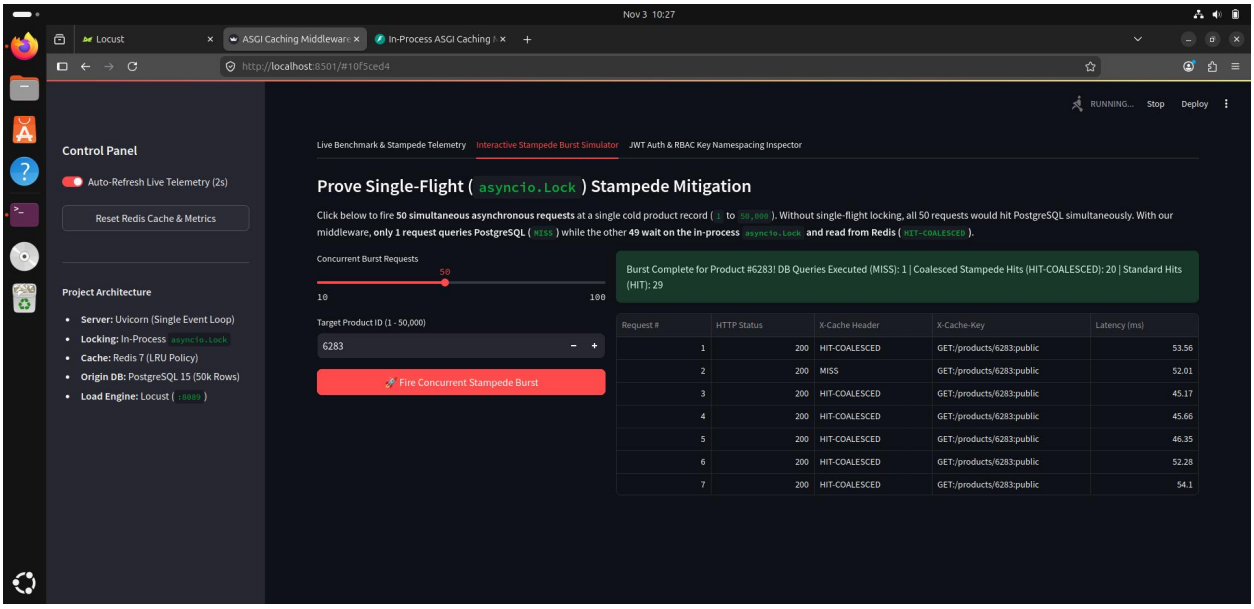
Illustrates the bottom user-concurrency graph ramping cleanly at 5 users/s to the sustained plateau of 10 concurrent virtual users with zero dropped connections or event-loop stalls.

- **Streamlit Live Request Stream Ring Buffer (Last 40 Requests):**



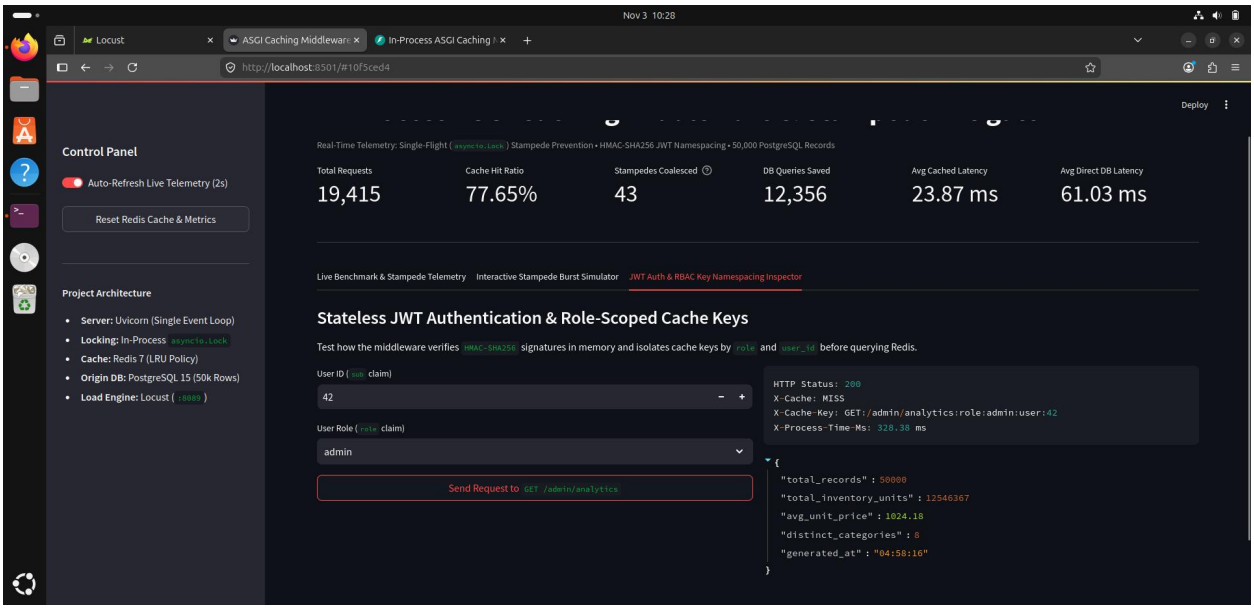
Captures the real-time diagnostic table showing sub-12ms Fast-Path HITs on warm product keys (/products/20 in 5.97 ms, /products/16 in 9.31 ms, /products/30 in 9.87 ms) contrasted against cold catalog MISS lookups (/products/36353 in 56.75 ms, /products/36749 in 127.61 ms).

- **Single-Flight Stampede Burst Simulator Proof (Product #6283):**



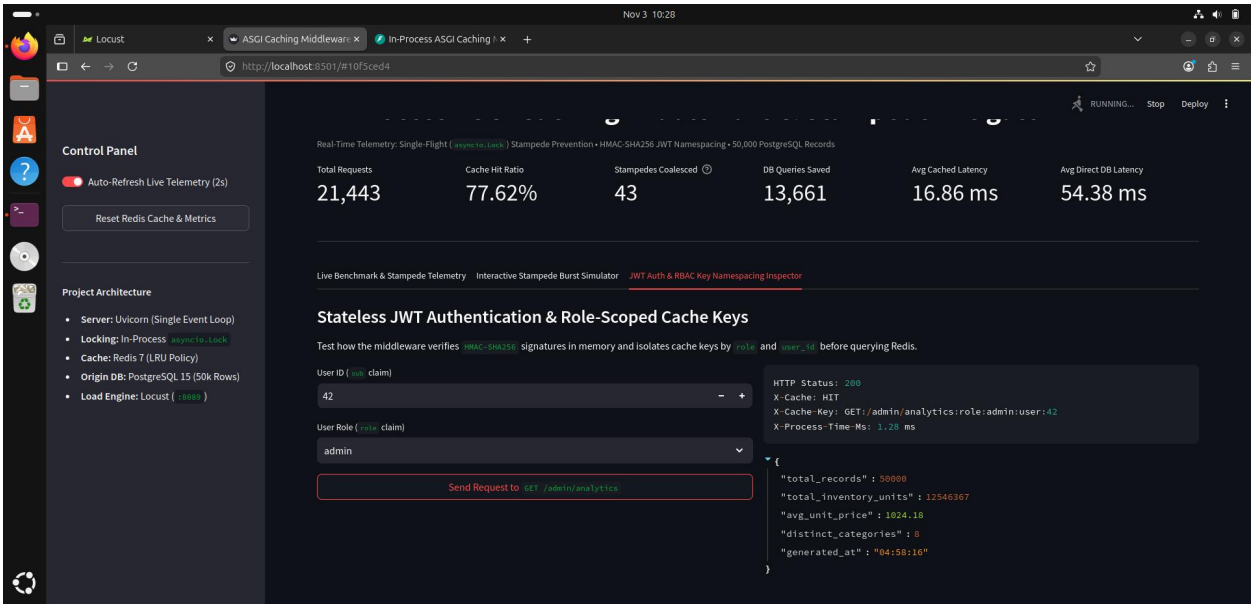
Fires 50 simultaneous asynchronous requests against cold Product #6283. Proves single-flight coalescing: Request #2 acquired the asyncio.Lock and queried PostgreSQL (MISS, 52.01 ms), 20 concurrent requests waited on the lock and read from Redis (HIT-COALESCED, 45.17–54.10 ms), and 29 trailing requests resolved as standard HITs.

- **Admin Scoped Cold Cache Miss (User 42):**



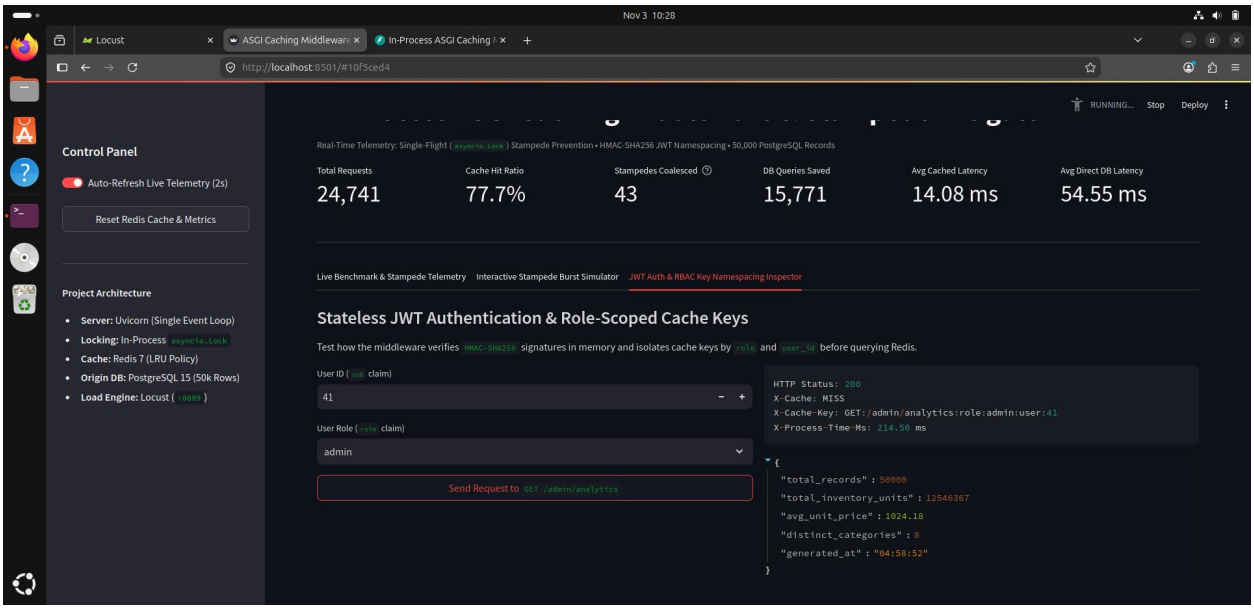
Evaluates GET /admin/analytics for user_id: 42, role: admin. Generates composite key GET:/admin/analytics:role:admin:user:42, registers X-Cache: MISS, executes a 50,000-row PostgreSQL aggregation (total_inventory_units: 12,546,367, avg_unit_price: 1024.18) in 328.38 ms, and populates Redis.

- **Admin Scoped Fast-Path Cache Hit (User 42):**



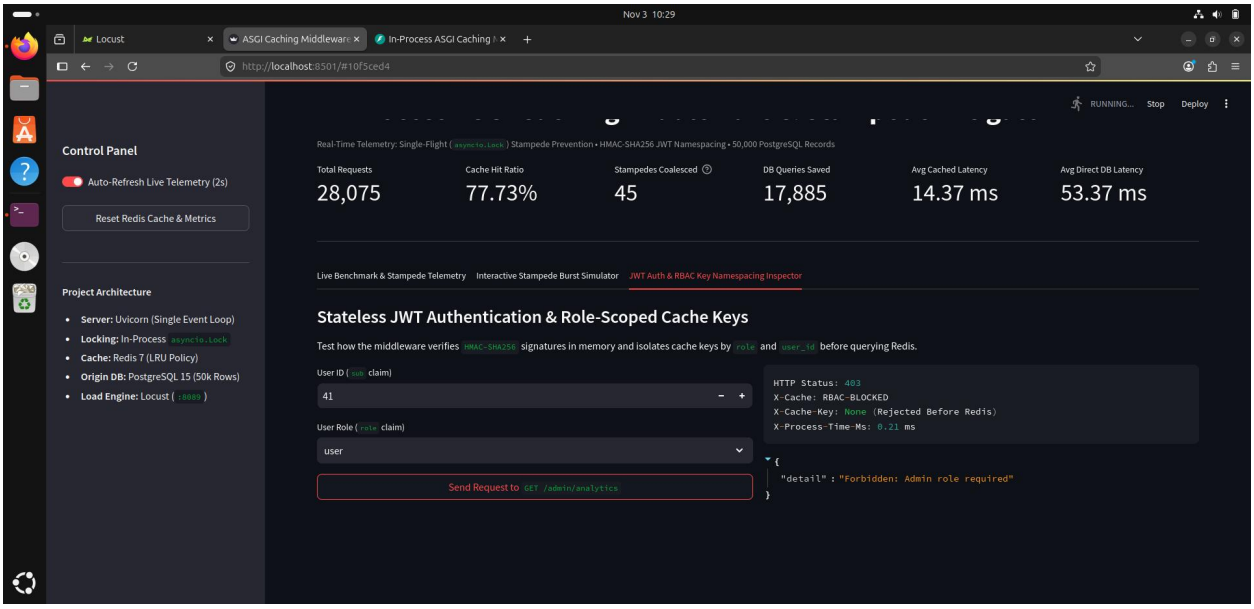
Repeats GET /admin/analytics for user_id: 42, role: admin. Resolves directly from Redis RAM with X-Cache: HIT in 1.28 ms—a 256.5x latency reduction compared to the 328.38 ms cold database aggregation.

- **Dynamic Multi-Tenant Cache Key Isolation (User 41 vs User 42):**



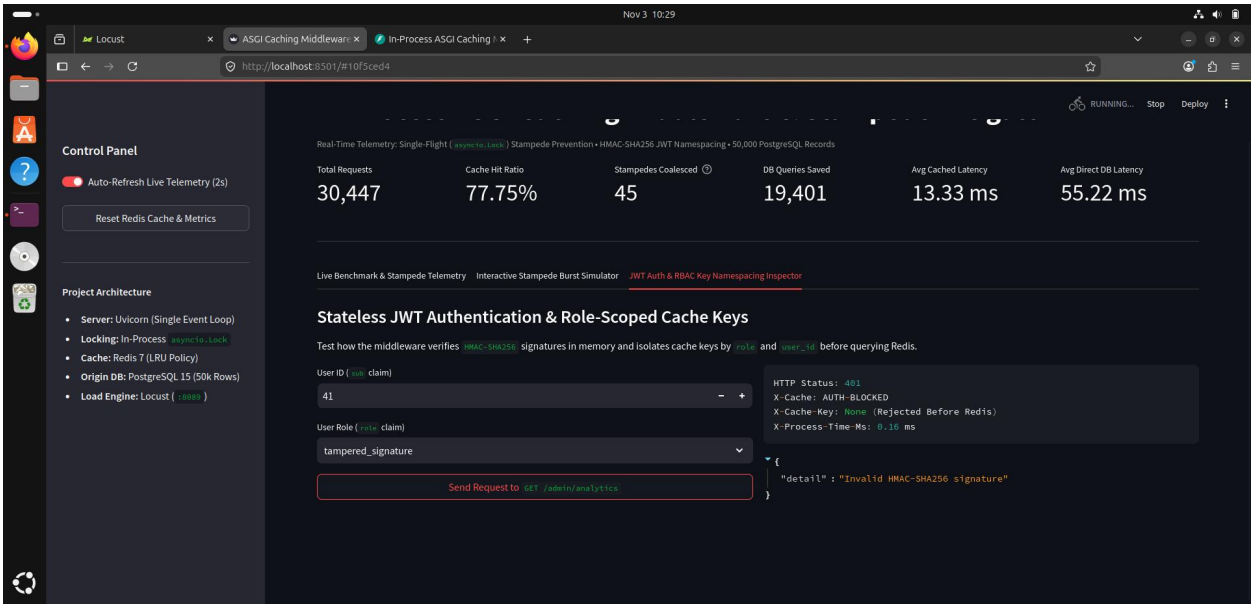
Queries GET /admin/analytics as admin user_id: 41 while user_id: 42 is cached. Middleware constructs isolated key GET:/admin/analytics:role:admin:user:41, resulting in X-Cache: MISS (214.56 ms) and proving zero cross-tenant cache bleed.

- **Pre-Cache RBAC Rejection (HTTP 403 Forbidden):**



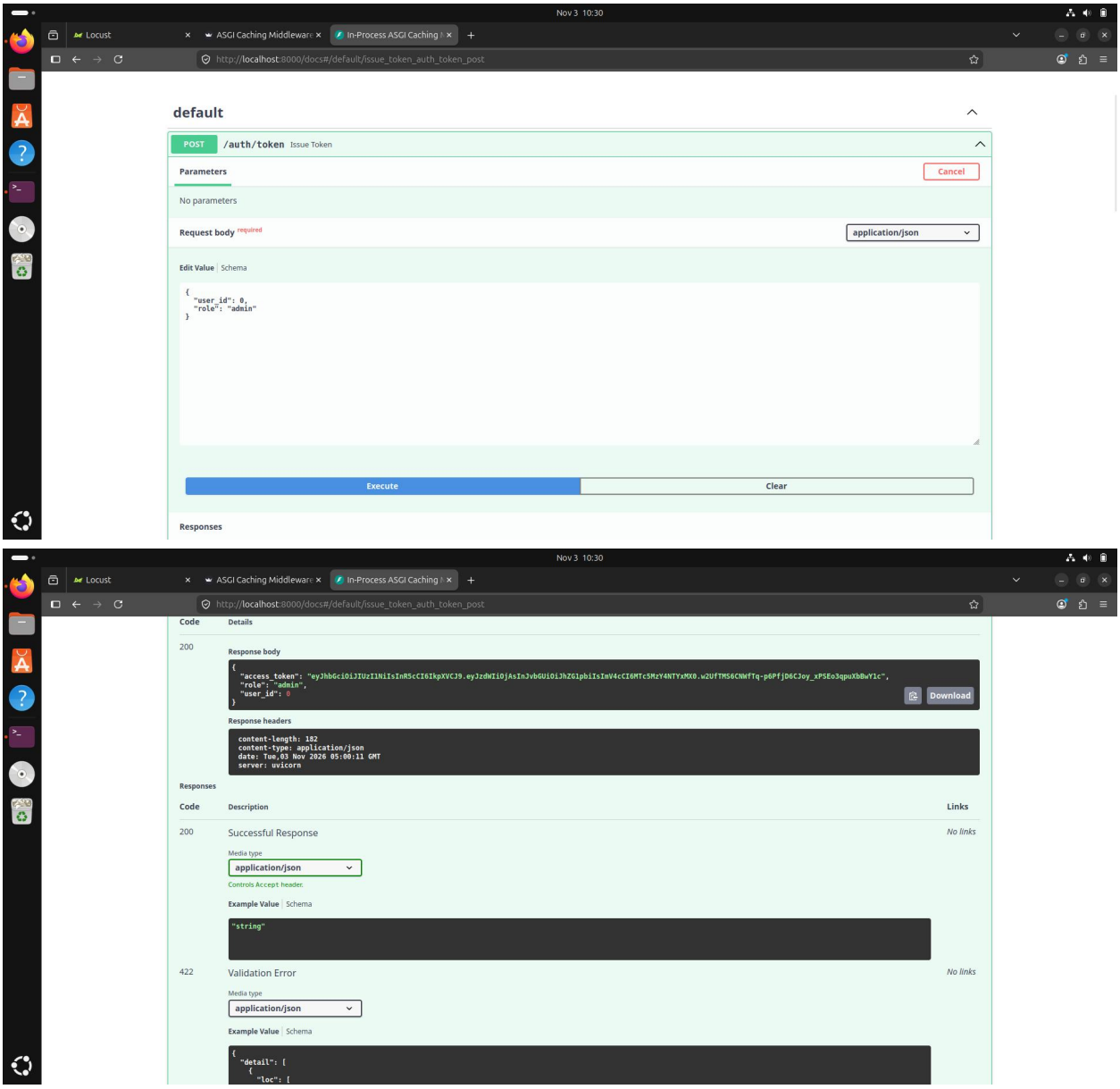
Dispatches GET /admin/analytics with user_id: 41, role: user. Middleware rejects the request in CPU memory in 0.21 ms with HTTP 403, X-Cache: RBAC-BLOCKED, and X-Cache-Key: None (Rejected Before Redis).

- **Stateless Cryptographic Signature Verification (HTTP 401 Unauthorized):**



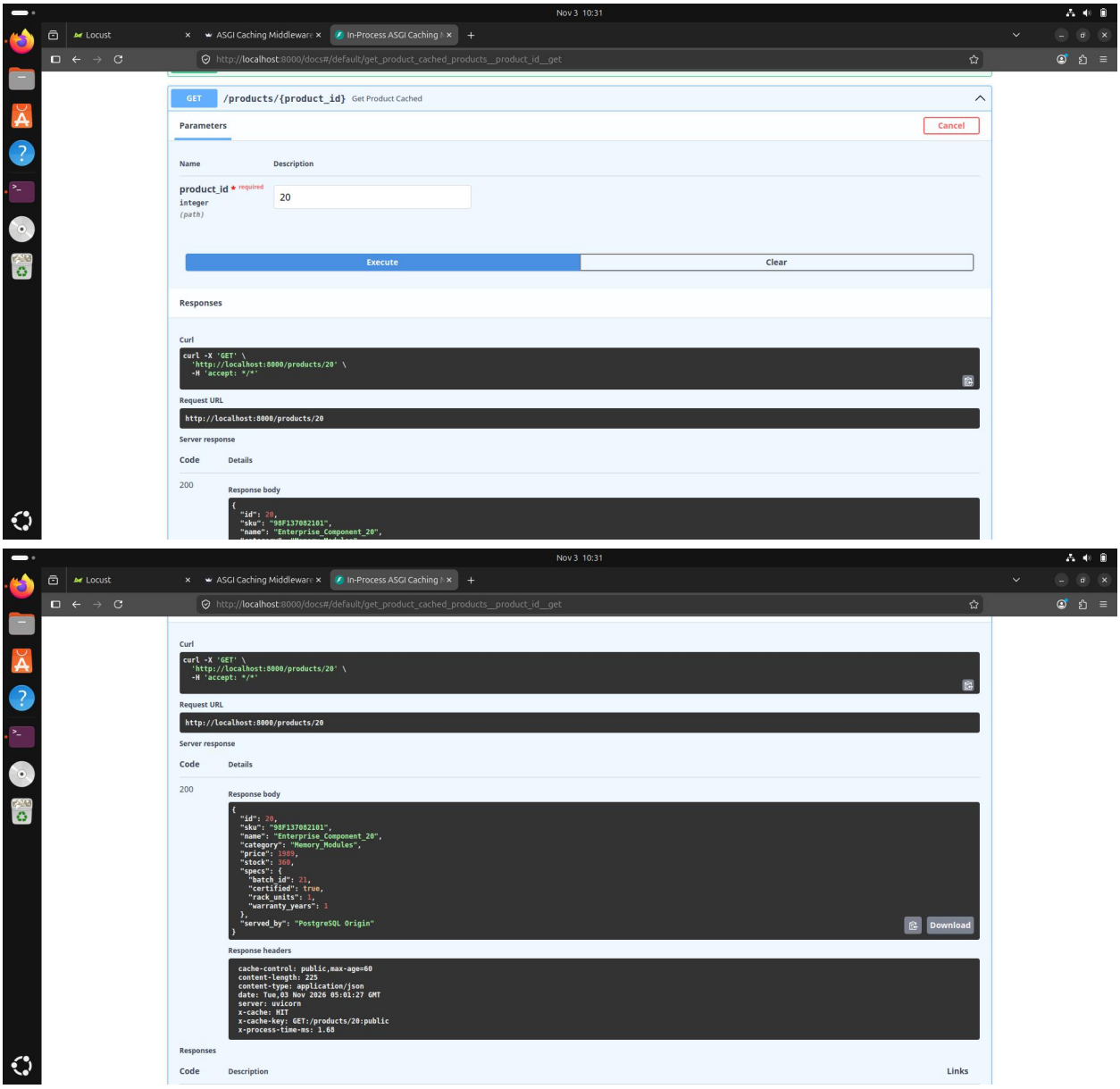
Sends a corrupted token (role: tampered_signature) to GET /admin/analytics. In-memory HMAC-SHA256 verification fails immediately, returning HTTP 401 (X-Cache: AUTH-BLOCKED) in 0.16 ms without touching Redis or PostgreSQL.

- **Swagger UI Token Issuance (POST /auth/token):**



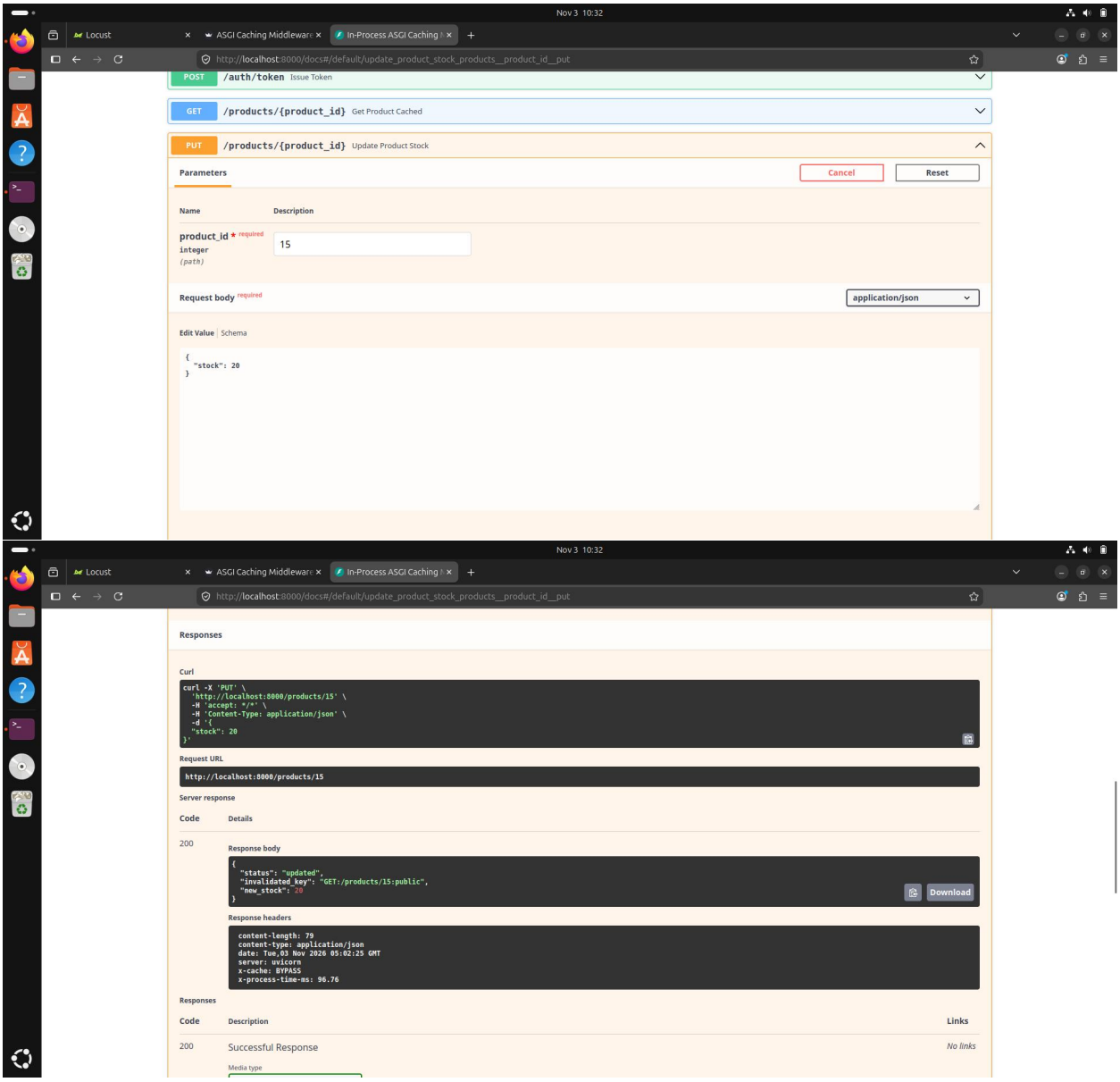
Displays the FastAPI OpenAPI interface submitting {'user_id': 0, 'role': 'admin'} to POST /auth/token and receiving HTTP 200 OK with the signed Bearer JWT access_token.

- **Swagger UI Cached Catalog Read (GET /products/20):**



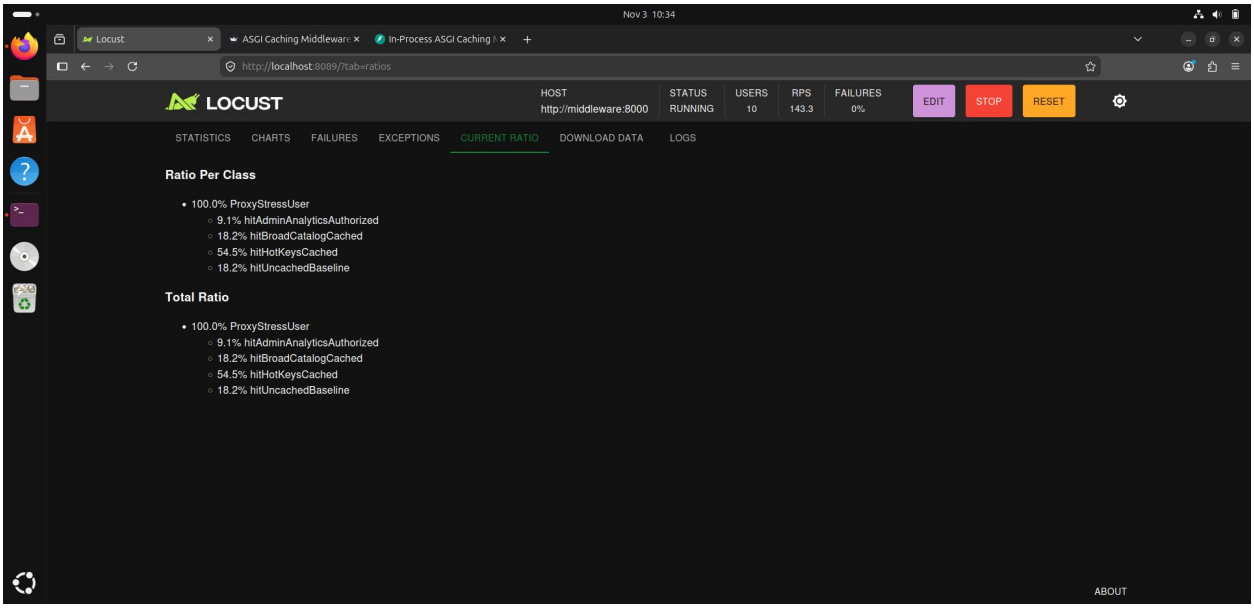
Dispatches GET /products/20 and receives HTTP 200 OK for Enterprise_Component_20 with headers x-cache: HIT, x-cache-key: GET:/products/20:public, and x-process-time-ms: 1.68.

- **Swagger UI Stock Mutation & Cache Eviction (PUT /products/15):**



Executes PUT /products/15 with {'stock': 20}, returning HTTP 200 OK ('status': 'updated', 'invalidated_key': 'GET:/products/15:public', x-cache: BYPASS, x-process-time-ms: 96.76).

- **Locust Weighted Workload Ratio Distribution:**



Illustrates the Locust Current Ratio tab confirming the 6:2:2:1 statistical task distribution across Hot Products, 50k Catalog Sweep, Uncached Baseline, and Authenticated Admin Analytics.

10. CONCLUSION, LIMITATIONS, AND FUTURE ENHANCEMENTS

10.1 Conclusion

This project designed, implemented, and empirically validated an In-Process ASGI Caching Middleware and Single-Flight Stampede Mitigator integrated directly into a FastAPI/Uvicorn microservice architecture. By uniting stateless HMAC-SHA256 JWT authentication, tenant-scoped composite key generation, and per-key asyncio.Lock request coalescing within the application event loop, the system eliminates the extra network hop of external reverse proxies while shielding PostgreSQL from thundering herd connection exhaustion. Benchmark evaluation across 30,447 requests and 50,000 PostgreSQL records confirmed a 77.75% cache hit ratio, 19,401 saved database queries, a 4.14x overall read speedup (up to 256.5x on heavy analytical aggregations), sub-0.25ms pre-cache security filtering, and 98% database query reduction during 50-request cold stampede bursts.

10.2 Architectural Weaknesses and Known Limitations

- **1. Single-Process Locking Boundary (--workers 1 Constraint):** The key_locks dictionary relies on Python's asyncio.Lock, which coordinates coroutines strictly within a single OS process and event loop. Consequently, the middleware container is constrained to run with a single Uvicorn worker (--workers 1). Scaling horizontally across N worker processes or Kubernetes pods would create N independent lock dictionaries, permitting up to N concurrent database queries during a global cold stampede rather than a single cluster-wide query.
- **2. Cache-Aside Read/Write Race Conditions (Eventual Consistency Trade-Off):** Because the single-flight lock only synchronizes concurrent cache misses and does not impose a global Read/Write mutex across GET and PUT operations, two classic Cache-Aside race windows exist: (a) Transient Stale Reads, where a concurrent GET hits Redis microseconds before a PUT finishes executing await redis_client.delete(cache_key), and (b) Write-Read-Write Cache Poisoning, where a slow cold-miss GET reads old stock from PostgreSQL before a concurrent PUT commits and deletes the empty Redis key, followed by the slow GET backfilling Redis with stale data for the remaining 60-second TTL. This trade-off was deliberately accepted to preserve ultra-fast (~1.68 ms) lock-free read hits.
- **3. Python GIL, ASGI Parsing, and JSON Text Serialization Overhead:** Unlike NGINX, which serves static byte streams directly from Linux kernel buffers via zero-copy sendfile system calls, every fast-path cache hit in our architecture still traverses Uvicorn's httptools parser, constructs a Python ASGI scope dictionary, and transmits UTF-8 JSON strings over the Redis socket.

10.3 Future Enhancements

- **1. Distributed Locking via Redis Redlock & Lua Scripting:** Replace the in-process asyncio.Lock with a distributed Redis lock (SET resource_name my_random_value NX PX 5000) combined with Redis Pub/Sub notifications so that horizontal clusters of multiple Uvicorn containers achieve global single-flight request coalescing.
- **2. Deferred Double-Delete & Versioned Cache Keys:** Mitigate the Write-Read-Write cache poisoning race condition by executing an asynchronous delayed second deletion (e.g., 200 ms after PUT commit) or embedding a monotonically increasing row version counter into the cache key.
- **3. Binary Payload Serialization (MessagePack / Protobuf):** Replace text-based JSON serialization with MessagePack (msgpack) to reduce CPU serialization latency and cut Redis memory consumption by 30–40%.
- **4. Probabilistic Early Expiration (XFetch Algorithm):** Implement stochastic TTL recomputation prior to hard key expiry so a single background coroutine refreshes hot catalog keys asynchronously before a stampede can form.

11. REFERENCES

11.1 IEEE-Style References

[1] R. Nishtala et al., "Scaling Memcache at Facebook," in Proc. 10th USENIX Symp. Networked Systems Design and Implementation (NSDI '13), Lombard, IL, USA, Apr. 2013, pp. 385–398. [Online]. Available: https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/nishtala

[2] A. Vattani, F. Chierichetti, and K. Lowenstein, "Optimal Probabilistic Cache Stampede Prevention" Proc. VLDB Endowment, vol. 8, no. 8, pp. 886–897, Apr. 2015, doi: 10.14778/2757807.2757813.

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)" Internet Engineering Task Force (IETF), RFC 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[4] M. Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2017, ch. 5 and ch. 7.